

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Modelos de Computação

Modelos formais de computação importantes para a compreensão das dificuldades inerentes de se resolver problemas computacionais.

- ▶ Máquina de Turing
 - ▶ λ -cálculo
 - ▶ RAM
 - ▶ Circuito booleano
- } modelos uniformes
- } modelo não uniforme

A especificação de um modelo de computação contempla

1. descrição do dispositivo (máquina/programa) do modelo,
2. como uma *entrada* é apresentada,
3. como a máquina computa, e apresenta o resultado,
4. uma noção de *passo elementar* de computação.

Máquina de Turing

é dada por uma sêxtupla

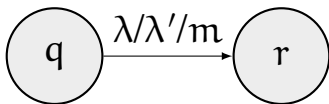
$$(Q, \Gamma, \Sigma, \delta, q_0, q_{\text{para}})$$

- ▶ Q é um conjunto finito de *estados*
- ▶ Γ é o *alfabeto da fita*
- ▶ δ é a *função de transição*
- ▶ $q_0 \in Q$ é o *estado inicial*
- ▶ $q_{\text{para}} \notin Q$ é o *estado final*

A entrada é dada numa fita de leitura/escrita.

A saída é escrita nessa fita.

As transições são os passos elementares.



se a máquina está no estado $q \in Q$ e lê $\lambda \in \Gamma$ na fita, então escreve $\lambda' \in \Gamma$ nessa posição e movimenta-se, de acordo com $m \in \{\leftarrow, \text{—}, \rightarrow\}$ ou uma posição para a esquerda, ou uma para a direita ou permanece na posição que está e, em seguida, entra no estado $r \in Q \cup \{q_{\text{para}}\}$.

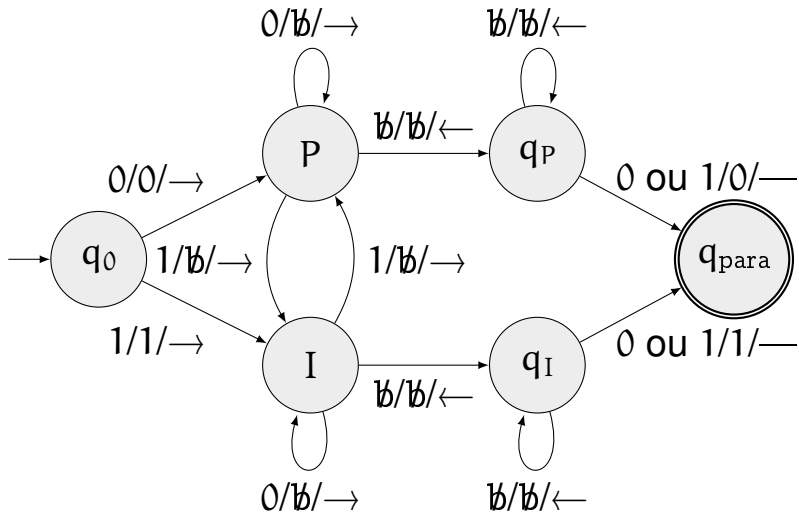
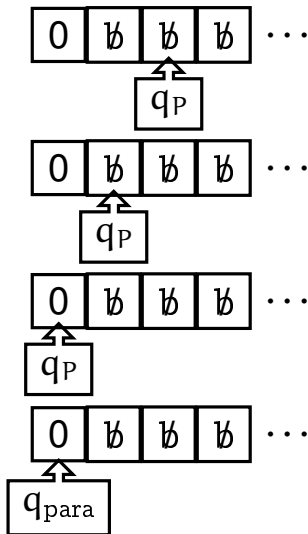
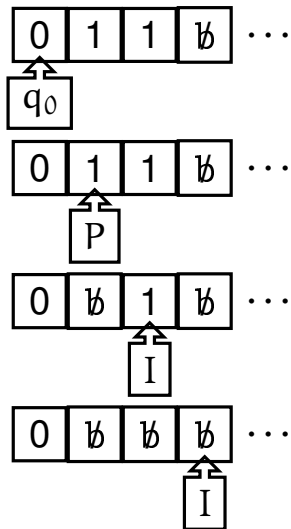


Diagrama de estados da máquina de Turing para o problema da paridade.



fita na execução para 'paridade' com 011.

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Problema computacional

$$(\mathcal{I}, \mathcal{O}, \mathcal{R})$$

- ▶ $\mathcal{I}$ denota o conjunto das instâncias (entradas) do problema,
- ▶ $\mathcal{O}$ o conjunto das respostas (saídas) e
- ▶ $\mathcal{R} \subset \mathcal{I} \times \mathcal{O}$ é uma relação que associa a cada instância uma ou mais respostas.

Uma *solução* de um problema computacional é um dispositivo de computação que computa uma função $A \subseteq \mathcal{R}$.

notação: o dispositivo C com instância $x \in \mathcal{I}$ responde $C(x) \in \mathcal{O}$.

As vezes precisamos distinguir x (entrada/saída) da sua *codificação* $\langle x \rangle$ usando símbolos de algum alfabeto, em geral consideramos bits.

Computabilidade um dado problema tem solução?

Complexidade quanto de um recurso
(tempo/espaco/sorteio) demanda uma
solução?

Complexidade de uma solução

Tempo de execução de pior caso: $T(n)$

Maior número de instruções básicas executadas por um dispositivo dentre todas as instâncias cujas descrição/codificação têm o mesmo comprimento n

Universalidade da MT

O conjunto de todas as máquinas de Turing é enumerável

$$M_1, M_2, \dots$$

É possível estabelecermos uma codificação binária canônica para as máquinas de Turing

Teorema

Existe uma máquina de Turing $\mathcal{U}$ ***universal*** tal que $\mathcal{U}(x, \langle n \rangle) = M_n(x)$. □

O conjunto de todas as máquinas de Turing é enumerável.

O conjunto de todas as funções $\mathbb{N} \rightarrow \{0, 1\}$ não é enumerável.

Logo tem funções que não são computáveis por MT. Dizendo outro modo, existem propriedades de números naturais que não são distinguíveis por programas.

Problema da Parada

decidir se a computação da MT M com uma entrada x termina não é computável.

Supõe existir MT H que recebe (i, x) e decide se a computação $M_i(x)$ termina

$$H(i, x) = \begin{cases} 0 & \text{se } M_i(x) \text{ não termina} \\ 1 & \text{se } M_i(x) \text{ termina} \end{cases}$$

Tome $P(x)$ a MT que computa como :

se $H(x, x) = 0$ **então responda** 1;
senão se $\mathcal{U}(x, x) = 1$ **então responda** 0;
senão responda 1.

Se $x := \langle n \rangle$ tal que $M_n = P$, então
 $\mathcal{U}(x, x) = M_n(\langle n \rangle) = P(x)$ e

$$P(x) = 0 \text{ se, e só se, } P(x) = 1$$

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Algoritmos eficientes

Modelo RAM tem registradores (memória), $M_0, M_1, \dots$, contador de programa, um conjunto de instruções para transferência de valores entre registradores

INC M_i	Incrementa o conteúdo do registrador i de 1
DEC M_i	Decrementa o conteúdo do registrador i de 1
CLR M_i	Substitui o conteúdo de M_i por 0
$M_i \leftarrow M_j$	Substitui o conteúdo de M_i pelo de M_j
$M_i \leftarrow MM_j$	Substitui o conteúdo de M_i pelo de M_{M_j}
$MM_i \leftarrow M_j$	Substitui o conteúdo de M_{M_i} pelo de M_j
JMP N_i	Atribui N_i ao contador de programa
$M_j \text{ JIFZ } N_i$	Se $M_j = 0$, atribui N_i ao contador de programa
CONTINUE	Continue na próxima instrução, caso exista, ou pare

Um *programa* RAM é uma sequência finita de instruções $N_0, N_1, \dots, N_m$

Equivalência polinomial

Uma MT que executa T transições pode ser simulada por um programa RAM que executa $O(T)$ instruções.

Um programa RAM com palavra de tamanho limitado que executa T instruções pode ser simulado por uma MT que executa $O(T^3)$ transições.

Codificações razoáveis de informação têm tamanho polinomialmente relacionados.

A classe das funções polinomiais é fechada para soma, multiplicação e composição, ou seja, *tempo polinomial* é preservado por práticas comuns de programação.

Computação eficiente é sinônimo de computação feita por um dispositivo com **tempo de execução polinomial** no tamanho da entrada.

O modelo formal é “irrelevante” com respeito a tempo polinomial.

Para simplificar nós dizemos, de modo genérico, **algoritmo de tempo polinomial** e usamos **pseudocódigo**.

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Linguagens

Instâncias e respostas são codificadas usando um alfabeto Σ (podemos fixar $\Sigma = \{0, 1\}$).

Uma *linguagem* L é um conjunto de *strings*/palavras sob Σ .

Uma MT M **decide** L se

$$M(x) = \begin{cases} 0 & \text{se } x \notin L \\ 1 & \text{se } x \in L \end{cases}$$

Exemplo de problema computacional descrito como linguagem:

Uma representação em Σ da fórmula booleana Φ é denotada por $\langle \Phi \rangle$ e a linguagem SAT é o conjunto

$$\text{SAT} = \{ \langle \Phi \rangle : \Phi \text{ é fórmula booleana satisfazível} \}.$$

Decidir se Φ é satisfazível é equivalente a decidir se a palavra $\langle \Phi \rangle$ pertence a linguagem formada pelas representações de fórmulas satisfazíveis.

Se L é linguagem, definindo (abusando de notação) também por L a função

$$L(w) = \begin{cases} 1 & \text{se } w \in L \\ 0 & \text{se } w \notin L \end{cases}$$

a MT M *decide* L se e só se M *computa* L .

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Classes de complexidade de tempo polinomial

A classe $\mathbb{P}$ — *Polynomial time* — é a classe das linguagens decididas por algoritmos de tempo polinomial.

Por exemplo, o problema de decidir se um inteiro positivo é primo está em $\mathbb{P}$.

Não sabemos se SAT pertence a $\mathbb{P}$ pois não conhecemos nenhum algoritmo de tempo polinomial para SAT.

A classe NP — *Nondeterministic Polynomial time* — é a classe das linguagens L para as quais existe um polinômio p e um algoritmo $M(\cdot, \cdot)$ de tempo polinomial tal que toda instância w

1. $w \in L$ então existe uma *entrada auxiliar* $c \in \{0, 1\}^{p(|w|)}$ tal que $M(w, c)$ responde *sim*;
2. se $w \notin L$ então para qualquer c o algoritmo $M(w, c)$ responde *não*.

$\text{SAT} \in \text{NP}$:

M com entrada (Φ, c) verifica em tempo polinomial se c é uma valoração das variáveis da fórmula booleana Φ que torna a fórmula verdadeira ou não.

Um algoritmo do tipo $M(w, c)$ é chamado de **verificador**.

Uma sequência c como acima é chamada de **certificado**.

Se $w \in L$ e $L \in \text{NP}$ então há um certificado curto que certifica tal fato, senão, caso $w \notin L$, não há um certificado curto de que $w \in L$.

$P \subseteq NP$:

se $L \in P$ e D é um algoritmo polinomial para L então
faça M para L que simula D e ignora a entrada
auxiliar. □

Conjectura de Cook–Levin

Principal problema não resolvido da Teoria da Computação e foi formulado independentemente por Stephen Cook e Leonid Levin em 1971

Problema

$$P \neq NP?$$

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

MT probabilística

MT probabilística M é uma máquina que sempre termina e que cada transição tem dois movimentos legais δ_0 e δ_1 , entretanto, só um deles é escolhido como o próximo passo. As escolhas δ_i são aleatória $i \in_{\mathbb{R}} \{0, 1\}$ e independentes.

O espaço de probabilidades é formado por todas as sequências binárias que representam as escolhas internas numa computação;

$$\mathbb{P}[M(w) = z]$$

é a soma das probabilidades de todas as sequências de escolhas aleatórias internas que terminam com a palavra $z \in \Sigma^*$ escrita na fita.

Uma MT probabilística M **decide** L **com probabilidade de erro** ϵ **em tempo** T se para toda palavra w

$$\mathbb{P}[M(w) \neq L(w)] \leq \epsilon$$

e $M(w)$ termina a computação em no máximo $T(|w|)$ transições para *qualquer que seja a sequência de bits aleatórios* usados na computação.

MT probabilística *off-line*

é uma MT determinística que tem uma entrada auxiliar escrita numa fita auxiliar só de leitura, além da entrada principal escrita na fita principal.

Essa fita auxiliar contém uma sequência B de bits aleatórios, cada posição tem um bit com distribuição uniforme e as ocorrências na sequência são independentes.

Cada posição da fita auxiliar é usada (lida) no máximo uma única vez e sob demanda.

O comprimento da entrada auxiliar pode ser definido como igual ao tempo T de execução da máquina.

Nesse caso, consideramos a distribuição da variável aleatória $M(w, B)$ da máquina probabilística M com entrada fixa w e um vetor aleatório $B \in_{\mathbb{R}} \{0, 1\}^{T(|w|)}$.

Uma MT probabilística *off-line* M com tempo de execução T **decide** L **com probabilidade de erro** ϵ se para toda palavra $w \in \Sigma^*$

$$\mathbb{P}_{B \in_{\mathbb{R}} \{0,1\}^{T(|w|)}} [M(w, B) \neq L(w)] \leq \epsilon.$$

Obs.: Os modelos são equivalentes.

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Classes probabilísticas

RP — *Randomized Polynomial time*

classe das linguagens L decididas por algoritmo probabilístico M de tempo polinomial tal que para algum polinômio p e para todo w

1. se $w \in L$, então

$$\mathbb{P}_{B \in_R \{0,1\}^{p(|w|)}} [M(w, B) \neq L(w)] \leq 1/3$$

2. se $w \notin L$, então

$$\mathbb{P}_{B \in_R \{0,1\}^{p(|w|)}} [M(w, B) \neq L(w)] = 0$$

A constante $\frac{1}{3}$ é uma escolha arbitrária, qualquer $\varepsilon \in (0, 1)$ resulta na mesma classe de problemas.
(Prova? — exercício)

Teorema

$$P \subseteq RP \subseteq NP.$$

Ideias da prova

- 1) para a 1ª inclusão: um algoritmo pra P pode ser convertido num pra RP , basta ignorar os bits aleatórios.
- 2) para a 2ª inclusão: podemos definir NP em termos de algoritmos probabilísticos

Podemos redefinir NP:
classe das linguagens L para as quais existe um polinômio p e um algoritmo probabilístico M de tempo polinomial tal que todo w

1. se $w \in L$ então

$$\mathbb{P}_{B \in_{\mathbb{R}} \{0,1\}^{p(|w|)}} [M(w, B) = L(w)] > 0,$$

2. se $w \notin L$ então

$$\mathbb{P}_{B \in_{\mathbb{R}} \{0,1\}^{p(|w|)}} [M(w, B) = L(w)] = 1$$



coRP — *Randomized Polynomial time*

classe das linguagens L decididas por algoritmo probabilístico M de tempo polinomial tal que para algum polinômio p e para todo w

1. se $w \in L$, então

$$\mathbb{P}_{B \in_R \{0,1\}^{p(|w|)}} [M(w, B) \neq L(w)] = 0$$

2. se $w \notin L$, então

$$\mathbb{P}_{B \in_R \{0,1\}^{p(|w|)}} [M(w, B) \neq L(w)] \leq 1/3$$

O que ocorre quando $L \in \text{RP} \cap \text{coRP}$?

algoritmo M_{RP} que prova $L \in \text{RP}$
+
algoritmo M_{coRP} que prova $L \in \text{coRP}$

algoritmo probabilístico que nunca erra

enquanto verdadeiro faça

se $M_{\text{RP}}(w) = 1$ **então responda** 1;

se $M_{\text{coRP}}(w) = 0$ **então responda** 0.

efeito colateral: pode levar muito tempo.

Um algoritmo probabilístico tem **tempo esperado** $T(n)$ se a variável aleatória t_w , que denota o tempo de execução do algoritmo com entrada w , tem valor esperado $\mathbb{E} t_w \leq T(|w|)$ para todo w .

A esperança é computada sobre os bits aleatórios usados na computação pelo algoritmo.

Nos casos em que T é um polinômio dizemos que a máquina é de *tempo esperado polinomial*. Por exemplo, o algoritmo aproximativo para o problema MAX-E3SAT

enquanto *verdadeiro* **faça**

se $M_{\text{RP}}(w) = 1$ **então** **responda** 1;

se $M_{\text{CORP}}(w) = 0$ **então** **responda** 0.

Qual a tempo esperado?

Se o algoritmo M_{RP} leva tempo $p(n)$ e se o algoritmo M_{CORP} leva tempo $q(n)$, então o tempo esperado é

$$\leq \frac{3}{2} \max\{p(n), q(n)\}$$

ZPP — *Zero-error Probabilistic Polynomial time*
classe das linguagens decididas por algoritmo
probabilístico M de tempo *esperado* polinomial tal
que

$$\mathbb{P}[M(w) = L(w)] = 1, \text{ para todo } w.$$

Lema

$$RP \cap coRP \subseteq ZPP$$

Equivalentemente,
 ZPP é a classe das linguagens para as quais existe um algoritmo probabilístico de tempo polinomial M tal que, para todo w ,

$$M(w) \in \{0, 1, ?\}$$

- ▶ 0 significa $w \notin L$
- ▶ 1 significa $w \in L$
- ▶ ? significa “não sei, parei em tempo polinomial” e $\mathbb{P}[M(w) = ?] \leq 1/2$.

Lema

$$\text{ZPP} = \text{RP} \cap \text{coRP}.$$

1. Só precisa provar $\subseteq$.
2. Tome L em ZPP , com alg. M de tempo esper. $p(n)$.
3. Prove L em RP .
4. Prove L em coRP .

L em RP

simula $M(w)$ até $\leq 3p(|w|)$ passos;
se $M(w) = 1$ **então responda** 1;
senão responda 0.

Se $w \notin L$ então $\mathbb{P}[\text{erro}] = 0$.

Se $w \in L$ então

$$\mathbb{P}[\text{erro}] = \mathbb{P}[t_w > 3p(|w|)] < \frac{p(|w|)}{3p(|w|)} = \frac{1}{3}$$

L em coRP

simula M com w até $\leq 3p(|w|)$ passos;
se $M(w) = 0$ **então responda** 0;
senão responda 1.

Se $w \in L$ então $\mathbb{P}[\text{erro}] = 0$.

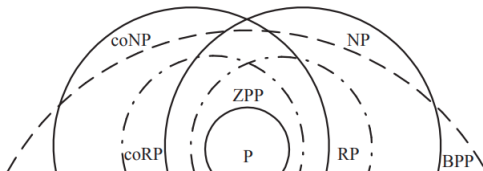
Se $w \notin L$ então

$$\mathbb{P}[\text{erro}] = \mathbb{P}[t_w > 3p(|w|)] < \frac{p(|w|)}{3p(|w|)} = \frac{1}{3}$$

BPP — *Bounded-error Probabilistic Polynomial time*
classe das linguagens decididas por algoritmos
probabilísticos de tempo polinomial com
probabilidade de erro

$$\mathbb{P}_{B \in_{\mathbb{R}} \{0,1\}^{p(|w|)}} [M(w, B) \neq L(w)] \leq 1/3.$$

A probabilidade de erro $1/3$ na definição não tem nada de especial, qualquer constante $\varepsilon \in (0, 1/2)$ serviria para definir a mesma classe de linguagens.



Problema

$BPP \subseteq P?$

Problema

$BPP \subseteq NP?$

Problema

$NP \subseteq BPP?$

Problema

Alguma(s) das inclusões em

$P \subseteq ZPP \subseteq (co)RP \subseteq BPP$

são próprias?

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Criptografia

Sigilo perfeito:

As codificações $E(P)$ e $E(P')$ deve ser *provavelmente* ($\mathbb{P}$) indistinguíveis.

Complexidade Computação:

As codificações $E(P)$ e $E(P')$ deve ser *efetivamente* (BPP) indistinguíveis.

Um sistema determinístico de criptografia não é seguro.

[Goldwasser, Shafi e Silvio Micali (1984). “Probabilistic encryption”.]

Primitivas criptográficas

$NP \not\subseteq P$ implica

1. uma função para a qual não é conhecido um algoritmo eficiente
2. conhecido um fato adicional (certificado), o problema tem um Verificador eficiente.

Primitivas criptográficas

$P \neq NP$ é uma afirmação de **pior caso**, não é suficiente para os propósitos da Criptografia. Tem que ser difícil na maioria das instâncias.

$$NP \not\subseteq BPP \Rightarrow NP \not\subseteq P$$

Primitivas criptográficas

1. Funções unidirecionais (*oneway function*).
2. Geradores pseudoaleatórios.

Função unidirecional

pode ser computada por algoritmo eficiente porém quase sempre não pode ser invertida por algoritmo eficiente

Função unidirecional

pode ser computada por algoritmo eficiente porém quase sempre não pode ser invertida por algoritmo eficiente

$$\text{MULT}(x, y) := \begin{cases} x \cdot y & \text{se } x, y \neq 1 \\ 1 & \text{caso contrário,} \end{cases}$$

Função unidirecional

pode ser computada por algoritmo eficiente porém quase sempre não pode ser invertida por algoritmo eficiente

$$\text{MULT}(x, y) := \begin{cases} x \cdot y & \text{se } x, y \neq 1 \\ 1 & \text{caso contrário,} \end{cases}$$

- fácil de computar
- as vezes difícil de inverter (obs $(1, xy) \notin f^{-1}(f(x, y))$)

$$\text{MULT}^{-1}(10.402.422.643)?$$



The number 10,402,422,643 is a prime number, meaning it cannot be factored into smaller whole numbers other than 1 and itself. Prime numbers are divisible only by 1 and the number itself, and they do not have any other divisors.

So, in this case, 10,402,422,643 is a prime number, and its only factors are 1 and 10,402,422,643.

$$10.402.422.643 = 331.319 * 31.397$$

Função unidirecional

$$f: \{0, 1\}^* \rightarrow \{0, 1\}^*$$

- ▶ $|f(x)|$ e $|x|$ polinomialmente relacionados;
- ▶ $f(x)$ computável em tempo polinomial em $|x|$;
- ▶ para todo algoritmo BPP M

$$\mathbb{P}_{\substack{X \in_R \{0,1\}^n \\ B \in_R \{0,1\}^*}} [M(f(X), B) \in f^{-1}(f(X))] < \frac{1}{n^d}$$

para todo $d \in \mathbb{N}$.

Função unidirecional

Problema

Funções unidirecionais existem?

Não sabemos. Se existir:

Função unidirecional

Problema

Funções unidirecionais existem?

Não sabemos. Se existir:

1. $NP \not\subseteq BPP$, consequentemente, $P \neq NP$
2. existem geradores pseudoaleatórios
3. existem provas conhecimento zero para NP
4. existem cripto. de chave privada seguros
5. existem esquemas de empenho de bits
6. existem provas naturais para $P \neq NP$

Candidatas a Função unidirecional

1. $MULT(x)$, x *string* binária, é multiplicação dos números representados pela metade mais significativa dos bits de x pela metade menos significativa dos bits de x
2. $x^k \bmod n$ (RSA, $k = 2$ significa inverter raiz quadrada)
3. Logaritmo discreto

Logaritmo discreto

– $(\mathbb{Z}_n, +_n)$:

$$\mathbb{Z}_n = \{0, 1, \dots, n-1\}$$

com soma módulo n ($+_n$) é grupo.

$+_3$	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

Logaritmo discreto

– $(\mathbb{Z}_n^*, \cdot_n)$:

$$\mathbb{Z}_n^* = \{ a \in \mathbb{Z}_n : \text{mdc}(a, n) = 1 \}$$

Com o produto módulo n $(\cdot_n)$ é grupo.

São os invertíveis ($a \cdot_n x = 1$ tem solução) módulo n .

$\cdot_3$	1	2
1	1	2
2	2	1

Logaritmo discreto

Se n é primo (ou potência de primo, ou ...) tem gerador $\alpha \in \mathbb{Z}_n^*$:

todo $b \in \mathbb{Z}_n^*$ é da forma α^k , algum $k \in \mathbb{N}$.

k é **logaritmo discreto** de b na base α .

É uma função

$$\text{dlog}_\alpha: (\mathbb{Z}_n^*, \cdot_n) \rightarrow (\mathbb{Z}_{\varphi(n)}, +_n)$$

$$\text{dlog}_\alpha: (\mathbb{Z}_p^*, \cdot_n) \rightarrow (\mathbb{Z}_{p-1}, +_n)$$

Logaritmo discreto

$$\mathbb{Z}_{11}^*$$

$$1 (= 2^0)$$

$$2 (= 2^1)$$

$$3 (= 2^8)$$

$$4 (= 2^2)$$

$$5 (= 2^4)$$

$$6 (= 2^9)$$

$$7 (= 2^7)$$

$$8 (= 2^3)$$

$$9 (= 2^6)$$

$$10 (= 2^5)$$

Logaritmo discreto

$\mathbb{Z}_{11}^*$	b	$\text{dlog}_2(b)$
1 ($= 2^0$)	1 ($= 2^0$)	0
2 ($= 2^1$)	2 ($= 2^1$)	1
3 ($= 2^8$)	3 ($= 2^8$)	8
4 ($= 2^2$)	4 ($= 2^2$)	2
5 ($= 2^4$)	5 ($= 2^4$)	4
6 ($= 2^9$)	6 ($= 2^9$)	9
7 ($= 2^7$)	7 ($= 2^7$)	7
8 ($= 2^3$)	8 ($= 2^3$)	3
9 ($= 2^6$)	9 ($= 2^6$)	6
10 ($= 2^5$)	10 ($= 2^5$)	5

Obs: 5 também é gerado no $\mathbb{Z}_{11}^*$.

Logaritmo discreto

Problema

Existe um algoritmo que dado um primo p determina em tempo polinomial um gerador no $\mathbb{Z}_p^*$?

Problema

Existe um algoritmo que dado b e α compute em tempo polinomial x tal que $b = \alpha^x \pmod{p}$?

Logaritmo discreto

Por exemplo, 3 é um gerador do $\mathbb{Z}_{1999}^*$.

Sabemos computar rapidamente $3^{789} \bmod 1999$
porém não sabemos resolver eficientemente a
equação $3^x = 1452 \bmod p$.

Logaritmo discreto

Por exemplo, 3 é um gerador do $\mathbb{Z}_{1999}^*$.

Sabemos computar rapidamente $3^{789} \bmod 1999$
porém não sabemos resolver eficientemente a
equação $3^x = 1452$ módulo p .

Dados p primo, α um gerador módulo p

$$f(p, \alpha, x) = (p, \alpha, \alpha^x)$$

pode ser calculada eficientemente mas não se sabe
inverter eficientemente (com contas mod p).

Diffie e Hellman, 1976

Troca de chaves: p e α públicos

- ▶ Alice sorteia $a \in \{2, 3, \dots, p - 2\}$, computa $A = \alpha^a \pmod{p}$ e envia para Bob;
- ▶ Bob sorteia $b \in \{2, 3, \dots, p - 2\}$, computa $B = \alpha^b \pmod{p}$ e envia para Alice;
- ▶ Alice computa B^a e Bob computa A^b e a chave *secreta* comum entre eles é

$$k = A^b = B^a = \alpha^{ab} \pmod{p}.$$

Tudo pode ser computado eficientemente.

Problema (DIFFIE–HELLMAN)

Existe um algoritmo eficiente que, dados p , α , A e B compute $\alpha^{ab} \bmod p$?

Problema

Um algoritmo eficiente para o Problema de Diffie–Hellman implica num algoritmo eficiente para o problema do logaritmo discreto?

O sistema Elgamal, 1985

Para criar as chaves pública e privada:

- ▶ sorteie um primo grande p e determine um gerador α de $\mathbb{Z}_p^*$;
- ▶ sorteie uniformemente $d \in \{2, 3, \dots, p - 2\}$ e compute $A = \alpha^d \bmod p$;
- ▶ *chave pública*: (p, α, A) ;
- ▶ *chave privada*: d (log discreto de A com base α).

A segurança do Elgamal é baseada na dificuldade computacional do *Problema de Diffie–Hellman*.

Elgamal — codificação

Com chave pública (p, α, A)

- ▶ dado o texto $P \in \mathbb{Z}_p$;
- ▶ sorteie uniformemente $b \in \{1, \dots, p-2\}$;
- ▶ compute $E_p(b, P) := (\alpha^b, A^b \cdot P)$.

Com as contas mod p . Tudo pode ser calculado eficientemente.

Elgamal — decodificação

Com chave pública (p, α, A) e privada d

- ▶ dado o texto cifrado (B, C)
- ▶ compute $D_d(B, C) = B^{p-1-d} \cdot C$.

Elgamal — decodificação

Com chave pública (p, α, A) e privada d

- ▶ dado o texto cifrado (B, C)
- ▶ compute $D_d(B, C) = B^{p-1-d} \cdot C$.

$$\begin{aligned} D_d(E_p(b, P)) &= D_d(\alpha^b, A^b \cdot P) \\ &= (\alpha^b)^{p-1-d} \cdot (\alpha^d)^b \cdot P \\ &= \alpha^{(p-1)b} \cdot P = P. \end{aligned}$$

Com as contas mod p . Tudo pode ser calculado eficientemente.

Assinatura digital Elgamal

Alice quer enviar um documento P com uma assinatura para Bob.

Assinatura digital Elgamal

Alice quer enviar um documento P com uma assinatura para Bob.

Sejam (p, α, A) a chave pública

- ▶ sorteia $k \in \{2, 3, \dots, p-2\} \cap \mathbb{Z}_{p-1}^*$,
- ▶ computa, usando sua chave privada,
 - $r = \alpha^k$ módulo p e
 - $s = (P - dr)k^{-1} \bmod p-1$
- ▶ envia $(P, (r, s))$ para Bob.

Para a verificação da assinatura o Bob verifica se

$$\alpha^P = A^r r^s \bmod p.$$

pois

$$\alpha^P = \alpha^{dr+sk} \bmod{p-1} = (\alpha^d)^r (\alpha^k)^s = A^r r^s$$